Rapport : Modélisation en Processing.

UNIVERSITÉ DE NAMUR

PROJET : GESTION D'UN TRAM AU COEUR DE MONTÉVEIL

Processing

INTRODUCTION :

Ce projet, réalisé dans le cadre du cours **INFOB236 – Projet de Programmation** de l'année académique 2024–2025, a pour objectif de modéliser et simuler la gestion d'un réseau de tramway dans la ville fictive de **Montéveil**.
Le contexte repose sur un environnement urbain contraint, avec des voies majoritairement uniques, où la coordination et la sécurité sont essentielles pour éviter toute collision et assurer la fluidité du trafic.

Le réseau est composé de **gares principales** et **gares secondaires**, reliées par des tronçons de voie unique et des sections de croisement équipées d'aiguillages. Les principales fonctionnalités modélisées dans notre application sont :

- Le suivi en temps réel de la **position des trams** sur le réseau.
- La gestion **des feux de signalisation** (rouge/vert) selon l'état d'occupation des voies.
- Le contrôle **des aiguillages** pour orienter les trams vers les voies libres.
- La mise en place de **procédures de sécurité** en cas de conflit de circulation.
- La représentation visuelle et interactive du réseau et des déplacements.
-

L'application a été développée en **Processing** à partir d'un modèle formel raffiné, en respectant les spécifications du projet.
Elle offre une interface graphique dynamique permettant de visualiser :

- Les gares et leurs connexions.
- Les trams en circulation ou à l'arrêt.
- L'état des infrastructures (feux, aiguillages, sections de voie).

Objectifs du projet

L'objectif principal de ce projet est de concevoir et de développer une application de simulation permettant de gérer le trafic d'un réseau de tramway en environnement contraint. Plus précisément, il s'agit de :

1. **Reproduire le fonctionnement d'un réseau réel** composé de gares principales et secondaires.
2. **Assurer la sécurité du trafic** en évitant toute collision entre trams.
3. **Gérer la signalisation et les aiguillages** en fonction de l'état d'occupation des voies.
4. **Informer en temps réel** sur la position et le déplacement des trams.
5. **Offrir une visualisation claire et intuitive** grâce à une interface graphique interactive développée en Processing.

Comment ouvrir et exécuter la simulation

1. **Décompresser le projet**
   Téléchargez l'archive du projet et **décompressez-la** dans un dossier de votre choix. Vérifiez que tous les fichiers `.pde` sont **dans le même dossier** (le "sketch" Processing) et que, s'il y a des images, elles se trouvent dans un sous-dossier **`data/`**.
2. **Installer Processing**
   Si Processing n'est pas installé, téléchargez la dernière version depuis le site officiel, installez-la puis **ouvrez Processing**.
3. **Ouvrir le projet**
   Dans Processing, cliquez sur **File > Open… (Fichier > Ouvrir…)**, puis sélectionnez le fichier **`redaprocessingaout.pde`** (c'est le fichier principal).
   → Processing ouvrira automatiquement les autres onglets (`Train.pde`, `Gare.pde`, `GareSecondaire.pde`, `Bande.pde`, `Deviation.pde`, `Aiguillage.pde`, `Feu.pde`, `Bouton.pde`, `dispatching.pde`, `Itineraire.pde`).
4. **Exécuter la simulation**
   Cliquez sur le bouton **Run** (icône ▶️) ou appuyez sur **Ctrl+R** (Windows/Linux) ou **Cmd+R** (macOS).
   Une fenêtre s'ouvre avec l'**affichage du réseau** (gares, voies, feux, aiguillages, trains).
5. **Voir les résultats et interagir**
   - Utilisez les **boutons** en bas de l'écran (ex. `G1→G2`, `G3→G1`) pour lancer des trajets.
   - **Cliquez sur un train** pour afficher ses informations dans le panneau **Dispatching** (en bas à droite).
   - Observez la mise à jour **automatique** des feux (rouge/vert) et des **aiguillages** selon l'occupation des voies.

Contexte et fonctionnement

Le réseau simulé est inspiré du cahier des charges fourni. La ville de Montéveil dispose d'un réseau composé de :

- **Gares principales** : points de convergence avec plusieurs voies (typiquement quatre), reliées entre elles par des lignes distinctes.
- **Gares secondaires** : situées entre deux gares principales, elles disposent d'une section de voie dédoublée pour permettre le croisement des trams.

Le fonctionnement repose sur plusieurs éléments clés :

- **Voies** : généralement uniques, elles imposent une gestion stricte des circulations dans les deux sens.
- **Feux de signalisation** : indiquent si la voie est libre (vert) ou occupée (rouge).
- **Aiguillages** : orientent les trams vers les voies libres ou déterminées par le plan de circulation.
- **Dispatching central** : coordonne en permanence les informations et les commandes (position des trams, état des feux, position des aiguillages).

En cas de conflit (tram en sens inverse ou voie occupée), des règles de priorité et des procédures de sécurité sont appliquées pour déterminer quel convoi doit reculer ou attendre à un point de croisement.

Description technique et architecture du programme

L'application a été développée en **Processing**, un langage et environnement basé sur Java, adapté à la création graphique et interactive.
Le programme est structuré en plusieurs fichiers `.pde`, chacun ayant un rôle précis dans la simulation.

Structure générale

dispatching.pde – Gestion et a2ichage du panneau de dispatching

Ce fichier implémente la classe **`Dispatching`**, responsable de l'interface de contrôle située en bas à droite de l'écran.
Il centralise les informations sur les trains et permet d'interagir avec eux via la souris.

Principales fonctionnalités :

- **Stockage et affichage d'informations** : le panneau affiche un titre et des lignes de texte décrivant l'état des trains ou du réseau.
- **Sélection des trains** : grâce à la méthode `onMousePressed()`, l'utilisateur peut cliquer sur un train pour afficher ses informations pendant un temps déterminé.
- **Gestion des coordonnées de référence** : la classe peut mémoriser une position d'ancrage (`gare2AnchorX`, `gare2AnchorY`) utile pour placer certains éléments de l'interface.
- **Interaction en temps réel** : un clic détecte la proximité d'un train par rapport à la position du curseur, en utilisant des zones de tolérance pour faciliter la sélection.

Rôle dans le projet :
Cette classe agit comme un **tableau de bord** du réseau, donnant une vue synthétique des trains en circulation et permettant à l'utilisateur de suivre et contrôler les éléments importants du trafic.

Train.pde – Modélisation et gestion des trains

Ce fichier définit la classe **`Train`**, qui représente chaque tram circulant sur le réseau. Elle gère à la fois l'affichage graphique du train, ses déplacements, ses interactions avec les infrastructures (gares, feux, aiguillages) et les règles de sécurité.

Principales caractéristiques :

- **Propriétés visuelles** : position (`x`, `y`), dimensions (`largeur`, `hauteur`) et image associée (`image_train`), avec un affichage par image ou par forme par défaut.
- **Gestion du mouvement** :
  - Déplacements sur courbes de Bézier (`tBezier`, `tBezier1`) pour simuler les trajets, y compris les sections en sens inverse.
  - Vitesse paramétrable (`vitesseBezier`).
  - Gestion des pauses avant entrée ou au milieu du trajet.
- **Interaction avec le réseau** :
  - Référence aux gares de départ et d'arrivée.
  - Attribution de voie (`voieAttribuee`).
  - Communication avec les feux et aiguillages.
- **Sécurité et gestion des collisions** :
  - Détection de collisions potentielles (`retourCollision`, `retourCollisiong3`, etc.). ○ Procédures de recul en cas de conflit (`estEnRecul`, `dejaEnRetour`).

- Délais d'attente et temporisations pour éviter les départs simultanés.

Rôle dans le projet :
La classe `Train` est l'élément central de la simulation. Elle **anime visuellement les trams** tout en appliquant les règles de circulation définies dans le cahier des charges.
Elle coordonne son comportement avec le **dispatching**, les **gares**, les **feux** et les **aiguillages** pour reproduire un trafic réaliste.

Gare.pde – Gestion des gares principales

Ce fichier définit la classe **`Gare`**, qui représente les gares principales du réseau.
Ces gares servent de points de départ, d'arrivée ou de croisement pour les trams, et peuvent accueillir plusieurs voies.

Principales caractéristiques :

- **Attributs** :
  - Coordonnées (`x`, `y`) et nom de la gare. o Listes pour gérer les trains présents (`train_en_gare`) et les trains dans des situations particulières (collision, retour, etc.). o Limite de quatre trains maximum par gare.
  - Indicateurs de blocage (`estBloqueeParCollision`) et de prévention des départs simultanés (`aLanceUnTrainCeFrame`).
- **Méthodes** :
  - `contientTrain(Train t)` : vérifie si un train se trouve physiquement dans la zone de la gare.
  - `spawn()` : permet de générer un nouveau train en gare s'il y a de la place.
  - Méthodes de gestion des départs, des arrivées et de la détection d'occupation.

Rôle dans le projet :
Les gares principales constituent les **nœuds stratégiques** du réseau, assurant la régulation des départs et des arrivées tout en respectant les contraintes de sécurité et de capacité.

GareSecondaire.pde – Gestion des gares secondaires

Ce fichier implémente la classe **`GareSecondaire`**, qui représente les gares intermédiaires situées entre deux gares principales.
Elles permettent le croisement de trams grâce à une courte section à double voie.

Principales caractéristiques :

- **Attributs** :
  - Coordonnées (`x`, `y`) et nom de la gare. o Largeur et longueur des voies, ainsi qu'une valeur `ampleur` pour définir l'espacement.
  - Liste `train_en_gare` pour suivre les trams présents.
- **Méthodes** :

- `afficher()` : dessine graphiquement la gare secondaire avec ses deux voies et son nom. o `moveAvecPause(Train t)` : gère le passage des trams avec un arrêt temporaire de 300 unités de temps pour simuler le croisement.

- **Rôle dans le projet** :
  Les gares secondaires permettent d'**éviter les collisions** sur les voies uniques en offrant un point de croisement temporaire, tout en régulant la vitesse et l'ordre de passage des trams.

Bande.pde – Représentation et gestion des segments de voie

Ce fichier définit la classe **`Bande`**, qui modélise un tronçon de voie reliant deux gares ou deux points du réseau.
Elle assure à la fois **l'affichage graphique** du segment et la **gestion du trafic** sur celui-ci.

Principales caractéristiques :

- **Attributs** :
  - Coordonnées (`x`, `y`) et nom du segment. o Dimensions (`width`, `length`, `ampleur`) pour représenter la largeur et l'espacement des rails.
  - Liste `train_en_bande` pour suivre les trains actuellement sur ce tronçon.
- **Méthodes** :
  - `afficher()` : dessine les rails avec deux lignes parallèles, des traverses en bois, et un texte descriptif.
  - `traverser_bande(Train t)` : gère le déplacement d'un train sur la voie, en ajoutant le train à la liste des trains présents puis en l'avançant jusqu'à la fin du segment.
  - `traverser_bande_retour(Train t)` : version utilisée pour les trajets en sens inverse, avec la même logique mais adaptée au mouvement de retour.
  - Mécanismes pour retirer le train de la liste lorsqu'il a quitté la bande, évitant ainsi les doublons ou les erreurs d'occupation.
- **Rôle dans le projet** :
  La classe `Bande` est essentielle pour **représenter les connexions physiques** entre les gares et assurer la gestion de l'occupation des voies.
  Elle joue un rôle clé dans la prévention des collisions en maintenant à jour la liste des trains présents sur chaque tronçon.

Aiguillage.pde – Gestion et a2ichage des aiguillages

Ce fichier définit la classe **`Aiguillage`**, qui représente les dispositifs permettant de rediriger un tram vers une voie différente à l'entrée d'une gare principale ou secondaire. Il joue un rôle essentiel dans la **sécurité** et la **fluidité** du trafic.

Principales caractéristiques :

- **Attributs** :
  - Coordonnées de départ (`xdep`, `ydep`) et d'arrivée (`xarr`, `yarr`) de l'aiguillage.
  - Dimensions (`longueur`, `largeur`) et couleur.

    - Orientation (`sens`) et position d'animation (`tAiguillage`).
- **Méthodes** :
    - `afficher()` : dessine l'aiguillage sous forme d'un segment avec une flèche indiquant la direction actuelle.
    - `dessinerFleche()` : calcule l'angle et dessine une flèche graphique montrant le sens de l'aiguillage.
    - `basculer(GareSecondaire g)` et `basculer1(Gare g)` : modifient la position de sortie de l'aiguillage en fonction de la voie libre détectée dans la gare correspondante.
- **Rôle dans le projet** :
  L'aiguillage est indispensable pour **orienter les trams vers les voies disponibles** et éviter les conflits de circulation.
  Il interagit directement avec les objets `Gare` et `GareSecondaire` pour adapter

la direction en fonction des conditions en temps réel.

Feu.pde – Gestion des feux de signalisation

Ce fichier définit la classe **`Feu`**, qui modélise les feux de signalisation présents le long du réseau de tram.
Ils servent à contrôler l'accès aux voies et à réguler le trafic pour éviter les collisions.

Principales caractéristiques :

- **Attributs** :
    - Coordonnées (`x`, `y`) de position du feu.
    - État (`estVert`) indiquant si le passage est autorisé (vert) ou interdit (rouge).
- **Méthodes** :
    - `afficher()` : dessine un feu rond (vert ou rouge) avec un support vertical. o `mettreVert()` et `mettreRouge()` : changent l'état du feu.
    - `mettreAJourSelonGare(Gare g)` : passe au rouge si la gare principale est pleine (≥ 4 trains), sinon au vert.
    - `mettreAJourSelonGares(GareSecondaire g)` : même logique adaptée aux gares secondaires (pleine si ≥ 2 trains).
- **Rôle dans le projet** :
  Les feux jouent un rôle essentiel dans la **sécurité du réseau**, en s'assurant qu'un train ne s'engage pas sur une voie déjà saturée.
  Ils interagissent directement avec les classes `Gare` et `GareSecondaire` pour mettre à jour leur état en temps réel.

Bouton.pde – Gestion des boutons d'interface

Ce fichier définit la classe **`Button`**, utilisée pour créer des boutons interactifs dans l'interface graphique de la simulation.
Ces boutons permettent d'effectuer certaines actions de contrôle, comme lancer ou arrêter un train, changer un aiguillage ou modifier des paramètres.

Principales caractéristiques :

- **Attributs** :
  - Position (`x`, `y`) et dimensions (`w`, `h`) du bouton.
  - Étiquette (`label`) affichée sur le bouton.
- **Méthodes** :
  - `display()` : dessine le bouton à l'écran avec un fond blanc, un contour noir et le texte centré.
  - `isClicked(int mx, int my)` : vérifie si un clic de souris s'est produit à l'intérieur des limites du bouton.
- **Rôle dans le projet** :
  Les boutons constituent la **partie interactive** de l'application, permettant à l'utilisateur d'intervenir directement sur la simulation.
  Ils offrent un moyen simple de tester et de contrôler certains éléments du réseau en temps réel.

Deviation.pde – Gestion des voies de déviation

Ce fichier définit la classe **`Deviation`**, qui représente un tronçon courbé du réseau, utilisé principalement pour les changements de direction ou les sections d'aiguillage.
Ces courbes servent aussi à gérer les croisements et manœuvres spéciales lorsque le réseau est encombré.

Principales caractéristiques :

- **Attributs** :
  - Coordonnées de départ (`x1`, `y1`) et d'arrivée (`x2`, `y2`). o Points de contrôle (`cx1`, `cy1`, `cx2`, `cy2`) pour dessiner une **courbe de Bézier** représentant la déviation.
  - Nom du tronçon.
  - Liste `train_en_dev` pour stocker les trains actuellement engagés sur la courbe.
- **Méthodes** :
  - `afficher()` : dessine la courbe en orange, affiche les trains présents dessus et le nom de la déviation.
  - `faireSuivreCourbe(Train train)` : fait avancer un train le long de la courbe en respectant certaines conditions :
    - Ralentissement ou arrêt si la gare suivante est saturée.
    - Ajustement de la vitesse (`vitesseBezier`) en fonction de la situation. ▪ Gestion des cas de collision et des retours.
- **Rôle dans le projet** :
  La classe `Deviation` permet de **gérer les courbes complexes du réseau**, en intégrant la logique de ralentissement, d'arrêt et de sécurité.
  Elle est particulièrement utile pour modéliser les changements de direction et les manœuvres d'évitement.

Itineraire.pde – Définition des trajets

Ce fichier définit la classe **`Itineraire`**, qui représente un trajet reliant une gare de départ à une gare de destination.
Il s'agit d'une structure de données simple mais essentielle pour organiser les déplacements des trains dans la simulation.

Principales caractéristiques :

- **Attributs** :
  - `depart` : référence à la gare de départ (`Gare`).
  - `destination` : référence à la gare d'arrivée (`Gare`).
- **Constructeur** : ○ Initialise l'itinéraire en stockant les gares de départ et de destination.
- **Variables globales** :
  - `trajetsActifs` : liste qui contient tous les itinéraires actuellement utilisés par les trains en circulation.

Rôle dans le projet :
La classe `Itineraire` permet de **modéliser et suivre les parcours** des trains.
Elle est utilisée pour savoir où un train doit aller et pour gérer l'état des trajets actifs sur le réseau.

redaprocessingaout.pde – Fichier principal et gestion de la simulation

Ce fichier contient le **code principal** du projet. Il relie toutes les classes définies dans les autres fichiers et orchestre le fonctionnement global de la simulation.
Il gère l'initialisation, le placement des éléments du réseau, la boucle de rendu et les interactions avec l'utilisateur.

Principales caractéristiques :

- **Déclarations et initialisation** :
  - Création des objets représentant les **gares principales** (`Gare`), **gares secondaires** (`GareSecondaire`), **bandes** (`Bande`), **déviations** (`Deviation`), **aiguillages** (`Aiguillage`) et **feux** (`Feu`).
  - Chargement des ressources graphiques (images des trains et gares).
  - Mise en place de l'interface `Dispatching` et des **boutons** (`Button`) permettant de déclencher des trajets spécifiques.
- **Boucle `setup()`** : ○ Définition de la taille de la fenêtre et du fond.
  - Initialisation des composants et positionnement des boutons.
- **Gestion des trajets** :
  - Tableau `trajets` définissant toutes les relations possibles entre gares (ex. `G1→G2`, `G3→G1`...).
  - Interaction avec les boutons pour lancer les trajets correspondants.
- **Boucle `draw()`** (présente dans le reste du fichier) :

- Mise à jour et affichage de tous les éléments : voies, gares, trains, feux, aiguillages, interface. ○ Application des règles de circulation (feux, collisions, basculement d'aiguillages).

- **Rôle dans le projet** :
  Ce fichier agit comme le **chef d'orchestre** du système. Il connecte tous les modules entre eux et contrôle la simulation en temps réel, garantissant la cohérence entre l'affichage et la logique métier.

Triggers du projet (événements qui déclenchent des actions)

---

1) Triggers "utilisateur / UI"

- **Clic sur un bouton** → lance un trajet prédéfini entre deux gares.
  Implémentation : `Button.isClicked(mx, my)` dans `Bouton.pde`, appelé depuis le fichier principal `redaprocessingaout.pde`.
- **Clic dans le panneau de dispatching** → sélectionne un train et affiche ses infos pendant un temps donné.
  Implémentation : `Dispatching.onMousePressed()` dans `dispatching.pde`. 2)

Triggers "infrastructure / sécurité"

- **Mise à jour automatique des feux** selon la capacité des gares :
  - **Gare principale pleine (≥ 4 trains)** → feu **rouge** ; sinon **vert**.
    Implémentation : `Feu.mettreAJourSelonGare(Gare g)` dans `Feu.pde`. ○ **Gare secondaire pleine (≥ 2 trains)** → feu **rouge** ; sinon **vert**.
    Implémentation : `Feu.mettreAJourSelonGares(GareSecondaire g)` dans `Feu.pde`.
- **Basculement d'aiguillage** quand une voie libre est détectée à l'entrée d'une gare (principale ou secondaire) → l'aiguillage oriente le tram vers la voie disponible.
  Implémentation : `Aiguillage.basculer(GareSecondaire g)` / `basculer1(Gare g)` dans `Aiguillage.pde`.

3) Triggers "état du réseau / trafic"

- **Entrée/sortie d'un tronçon de voie (bande)** → ajout/suppression du train dans la file d'occupation du tronçon.
  Implémentation : `Bande.traverser_bande(Train t)` et `traverser_bande_retour(Train t)` dans `Bande.pde`.
- **Entrée sur une courbe (déviation)** → ajout du train dans `train_en_dev` + suivi de la courbe de Bézier ; **sortie** → retrait de la liste et reprise du trajet linéaire.
  Implémentation : `Deviation.faireSuivreCourbe(Train train)` dans `Deviation.pde`.

- **Gare pleine** → déclenche un **arrêt/attente** du train en amont (vitesse réduite ou pause) jusqu'à libération.
  Implémentation : logique de pause/temporisation dans `Train.pde` et `Deviation.pde` (ajustement de `vitesseBezier`, flags d'attente).

4) Triggers "mouvement et sécurité des trains"

- **Collision potentielle / conflit de sens** → activation de flags de sécurité et **recul** si nécessaire.
  Implémentation : multiples drapeaux dans `Train.pde` (`retourCollision...`, `estEnRecul`, `dejaEnRetour`) + logique associée.
- **Arrivée à un point clé (coordonnées de contrôle)** → changement d'état : début/fin de tronçon, passage de bande à déviation, entrée en gare, etc.
  Implémentation : tests de position (`x`, `y`) dans `Train.pde` et `Deviation.pde` (ex. égalité avec `x1,y1`, `x2,y2`).
- **Départ autorisé** (feu vert + voie libre + capacité disponible) → lancement/continuation du mouvement (incrément de `tBezier` / `vitesseBezier`).
  Implémentation : `Train.pde` (moteur de déplacement) et contrôles depuis `Feu.pde` / `Gare*.pde`.

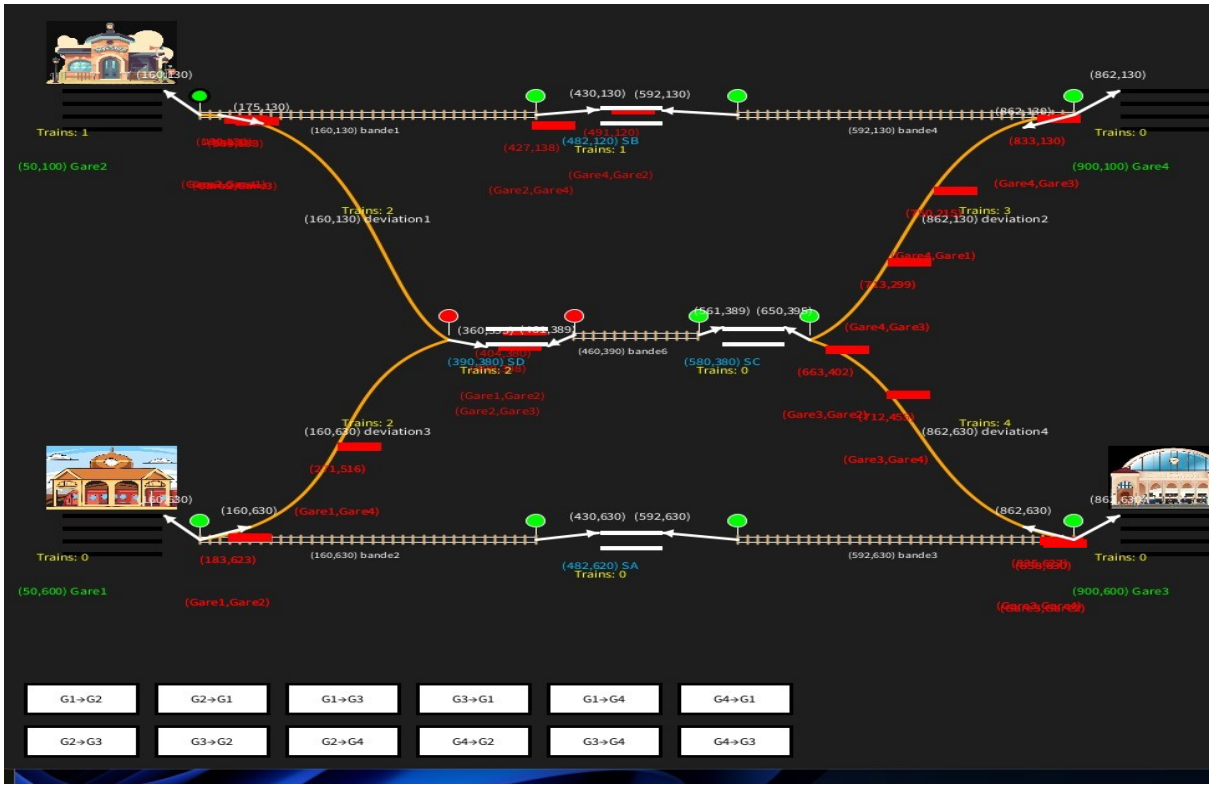
5) Triggers "gestion des gares"

- **Détection de présence d'un train dans la zone gare** → mise à jour de la liste des trains en gare, prévention des départs simultanés.
  Implémentation : `Gare.contientTrain(t)` + gestion de `train_en_gare`, `aLanceUnTrainCeFrame`, `estBloqueeParCollision` dans `Gare.pde`.
- **Spawn d'un train (si place disponible)** → ajout d'un nouveau train en gare (limite **4** en principale).
  Implémentation : `Gare.spawn()` dans `Gare.pde`.

6) Triggers "orchestration globale"

- **Initialisation** (`setup()`) → création des gares, bandes, déviations, feux, aiguillages, boutons, chargement des images.
  Implémentation : `redaprocessingaout.pde`.
- **Boucle temps réel** (`draw()`) → rafraîchit l'affichage, met à jour feux/aiguillages, avance les trains, gère collisions et états d'attente.
  Implémentation : `redaprocessingaout.pde`

Résultats et aDichage final:



L'exécution de l'application *Processing* produit une **interface visuelle interactive** qui représente fidèlement le réseau de tram de la ville fictive de **Montéveil**. L'écran se compose de plusieurs zones et éléments dynamiques :

1. Infrastructures du réseau

- **Gares principales** :
  - Situées aux quatre coins du réseau (`Gare1`, `Gare2`, `Gare3`, `Gare4`).
  - Chaque gare principale peut accueillir jusqu'à **4 trains simultanément**.
  - Affichées avec une image représentative (bâtiment) et le nom en vert.
- **Gares secondaires** :
  - Placées au centre des segments reliant les gares principales. ○ Dotées de **deux voies parallèles** pour permettre le croisement de trams.
  - Capacité maximale de **2 trains**.
- **Voies droites (Bandes)** :
  - Segments ferroviaires horizontaux, tracés en blanc avec traverses marron. ○ Chaque bande est nommée (`bande1`, `bande2`, etc.) et affiche l'état d'occupation.
- **Voies courbes (Déviations)** :
  - Représentées en **orange**, elles permettent de passer d'une ligne à une autre ou de contourner une gare. ○ Utilisent des courbes de Bézier pour un rendu fluide.

2. Éléments de contrôle et sécurité

- **Feux de signalisation** :

    - Points ronds rouges ou verts placés à l'entrée des voies. ○ Vert = voie libre, Rouge = voie occupée ou gare pleine.
    - Mis à jour automatiquement selon l'état des gares et des voies.
- **Aiguillages** :
    - Flèches indiquant la direction dans laquelle un tram sera dirigé. ○ Changent automatiquement de position selon la voie disponible.

3. Circulation et trains

- Les trams sont représentés par de petites formes (ou images) se déplaçant le long des voies.
- Les déplacements suivent les règles suivantes :
    - **Vitesse constante** sur les voies libres. ○ **Ralentissement ou arrêt** si le tronçon suivant est occupé. ○ **Arrêt prolongé** si la gare suivante est pleine (feu rouge).
    - **Recul** possible en cas de conflit de sens (collision évitée).
- Chaque déviation et bande affiche le **nombre de trains en cours de traversée**.

4. Interface utilisateur

- **Boutons interactifs** (en bas de l'écran) :
    - Permettent de lancer un trajet spécifique entre deux gares (ex. `G1→G2`, `G3→G1`).
- **Panneau Dispatching** (en bas à droite) :
    - Affiche des informations sur un train sélectionné par clic. ○ Montre des détails en temps réel pendant quelques secondes.

5. Fonctionnalité et cohérence

L'affichage permet :

- De visualiser en un coup d'œil **l'état du réseau** (occupations, positions des trains, feux et aiguillages).
- De **tester différents scénarios** de circulation via les boutons.
- De vérifier la **réactivité des éléments** (feux, aiguillages) face aux changements de trafic.